```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam

tf.random.set_seed(42)
np.random.seed(42)

df=pd.read_csv("mnist_784_csv.csv")
df.shape
```

```
(70000, 785)
```

```
x=df.iloc[:,:-1].values.astype("float32").reshape(-1,28,28)
y=df.iloc[:,-1].values.astype("float32")

print("Images", x.shape, " Labels", y.shape, " Classes", np.unique(y))
```

```
Images (70000, 28, 28)  Labels (70000,)  Classes [0. 1. 2. 3. 4. 5. 6.
7. 8. 9.]
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,rando
m_state=42)

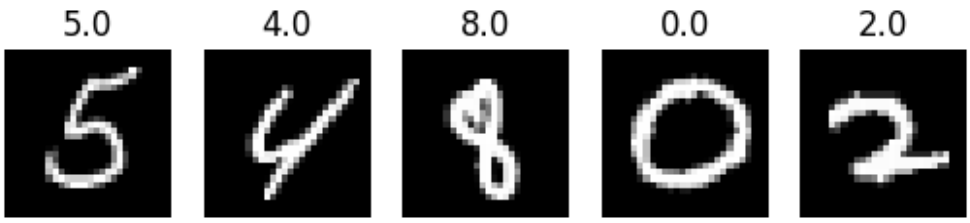
x_train=x_train/255.0
x_test=x_test/255.0

y_train_sc=to_categorical(y_train,10)
y_test_sc=to_categorical(y_test,10)

print("train ",x_train.shape)
print("test",x_test.shape)
```

```
train  (56000, 28, 28)
test (14000, 28, 28)
```

```
for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(y_train[i])
    plt.axis("off")
plt.show()
```




```
model=keras.Sequential([
    layers.Input(shape=(28,28)),
    layers.Flatten(),
    layers.Dense(256,activation="relu"),
    layers.Dense(128,activation="relu"),
    layers.Dense(10,activation="softmax"),
])
model.summary()
```

```
Model: "sequential_1"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃       Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ flatten_1 (Flatten)             │ (None, 784)            │             0 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_3 (Dense)                 │ (None, 256)            │       200,960 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_4 (Dense)                 │ (None, 128)            │        32,896 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_5 (Dense)                 │ (None, 10)             │         1,290 │
└─────────────────────────────────┴────────────────────────┴───────────────┘
```

```
 Total params: 235,146 (918.54 KB)

 Trainable params: 235,146 (918.54 KB)

 Non-trainable params: 0 (0.00 B)
```

```
model.compile(
    loss="categorical_crossentropy",
    optimizer=Adam(learning_rate=0.001),
```

```
    metrics=["accuracy"],
)

history=model.fit(
    x_train,
    y_train_sc,
    batch_size=128,
    epochs=20,
    validation_split=0.1,
    verbose=2
)
```

```
Epoch 1/20
394/394 - 4s - 10ms/step - accuracy: 0.9143 - loss: 0.3006 -
val_accuracy: 0.9616 - val_loss: 0.1403
Epoch 2/20
394/394 - 1s - 3ms/step - accuracy: 0.9656 - loss: 0.1157 -
val_accuracy: 0.9716 - val_loss: 0.0978
Epoch 3/20
394/394 - 1s - 3ms/step - accuracy: 0.9781 - loss: 0.0732 -
val_accuracy: 0.9754 - val_loss: 0.0824
Epoch 4/20
394/394 - 1s - 3ms/step - accuracy: 0.9855 - loss: 0.0509 -
val_accuracy: 0.9755 - val_loss: 0.0783
Epoch 5/20
394/394 - 1s - 3ms/step - accuracy: 0.9902 - loss: 0.0361 -
val_accuracy: 0.9761 - val_loss: 0.0791
Epoch 6/20
394/394 - 1s - 3ms/step - accuracy: 0.9920 - loss: 0.0275 -
val_accuracy: 0.9732 - val_loss: 0.0859
Epoch 7/20
394/394 - 1s - 3ms/step - accuracy: 0.9934 - loss: 0.0228 -
val_accuracy: 0.9782 - val_loss: 0.0758
Epoch 8/20
394/394 - 1s - 3ms/step - accuracy: 0.9946 - loss: 0.0185 -
val_accuracy: 0.9805 - val_loss: 0.0762
Epoch 9/20
394/394 - 1s - 3ms/step - accuracy: 0.9951 - loss: 0.0158 -
val_accuracy: 0.9775 - val_loss: 0.0880
Epoch 10/20
394/394 - 1s - 3ms/step - accuracy: 0.9961 - loss: 0.0125 -
val_accuracy: 0.9779 - val_loss: 0.0907
Epoch 11/20
394/394 - 1s - 3ms/step - accuracy: 0.9964 - loss: 0.0113 -
val_accuracy: 0.9770 - val_loss: 0.0862
Epoch 12/20
394/394 - 1s - 3ms/step - accuracy: 0.9959 - loss: 0.0123 -
val_accuracy: 0.9787 - val_loss: 0.0919
Epoch 13/20
394/394 - 1s - 3ms/step - accuracy: 0.9961 - loss: 0.0117 -
```

```
val_accuracy: 0.9782 - val_loss: 0.0872
Epoch 14/20
394/394 - 1s - 3ms/step - accuracy: 0.9973 - loss: 0.0086 -
val_accuracy: 0.9768 - val_loss: 0.1028
Epoch 15/20
394/394 - 1s - 3ms/step - accuracy: 0.9972 - loss: 0.0086 -
val_accuracy: 0.9730 - val_loss: 0.1270
Epoch 16/20
394/394 - 1s - 3ms/step - accuracy: 0.9965 - loss: 0.0100 -
val_accuracy: 0.9805 - val_loss: 0.0953
Epoch 17/20
394/394 - 1s - 3ms/step - accuracy: 0.9974 - loss: 0.0076 -
val_accuracy: 0.9796 - val_loss: 0.1023
Epoch 18/20
394/394 - 1s - 3ms/step - accuracy: 0.9972 - loss: 0.0078 -
val_accuracy: 0.9773 - val_loss: 0.1194
Epoch 19/20
394/394 - 1s - 3ms/step - accuracy: 0.9976 - loss: 0.0070 -
val_accuracy: 0.9787 - val_loss: 0.1107
Epoch 20/20
394/394 - 1s - 3ms/step - accuracy: 0.9987 - loss: 0.0047 -
val_accuracy: 0.9791 - val_loss: 0.1021
```

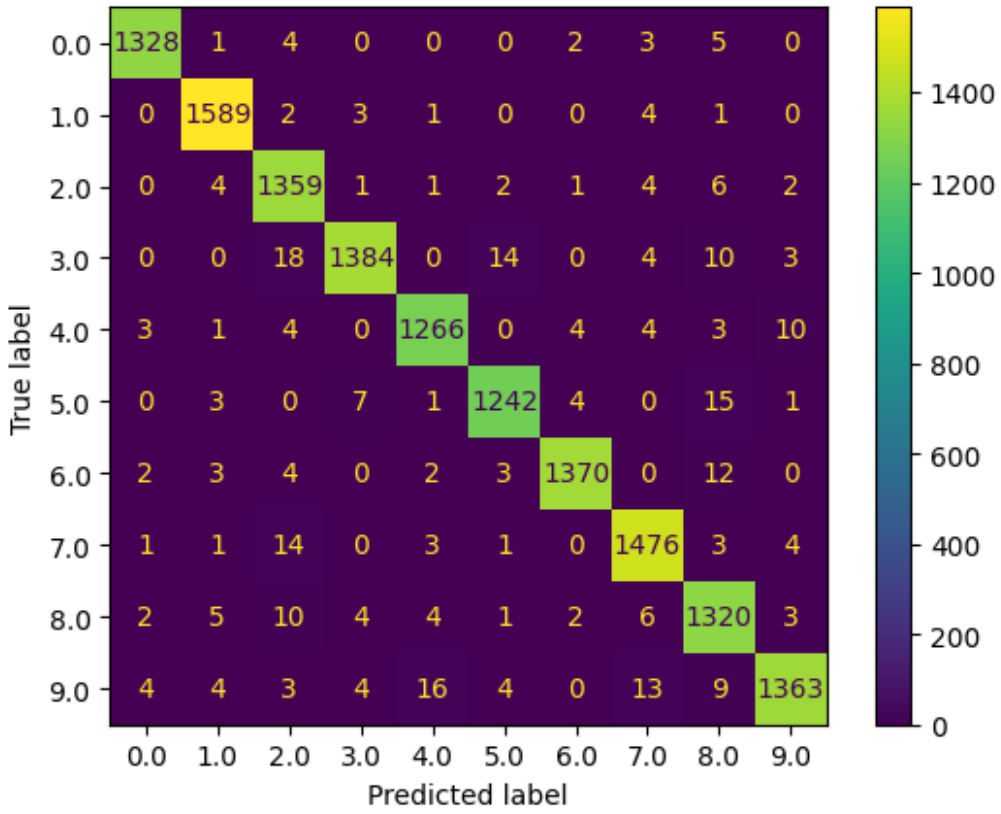
```
test_loss, test_acc=model.evaluate(x_test,y_test_sc)
print(test_loss,"|" ,test_acc)

y_pred=model.predict(x_test).argmax(axis=1)
```

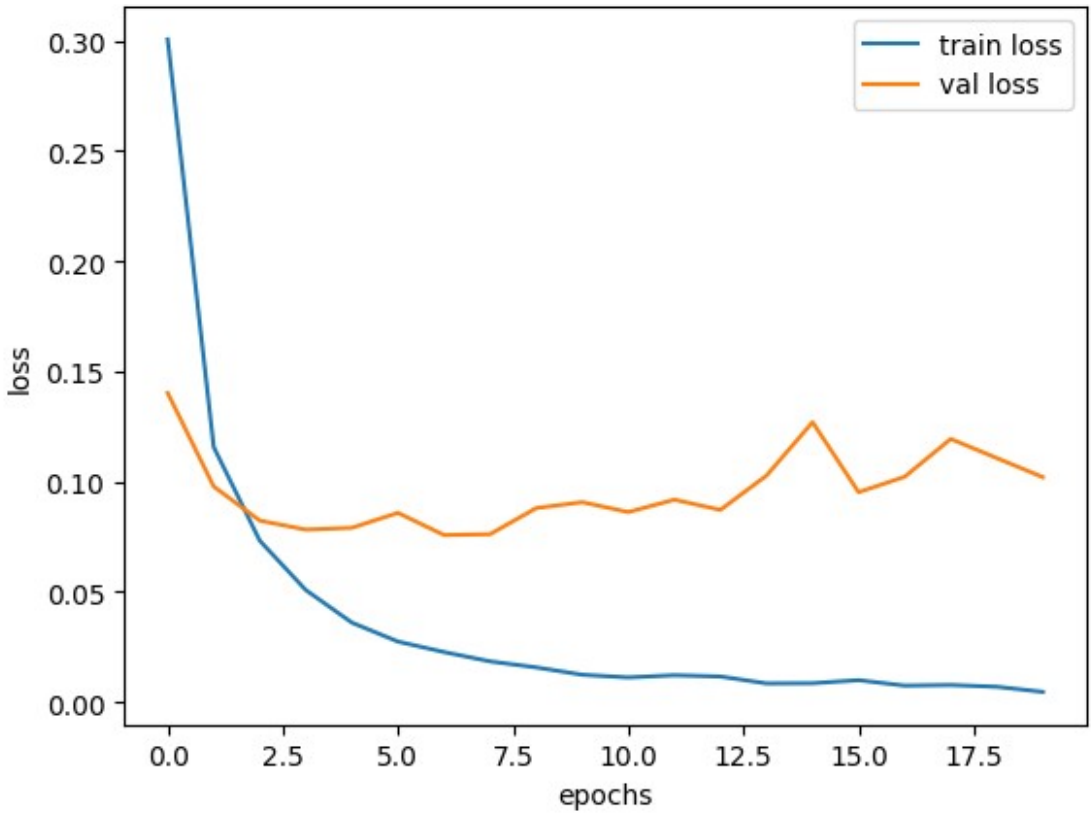
```
438/438 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9784 - loss:
0.1297
0.1296817660331726 | 0.9783571362495422
438/438 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```

```
print(confusion_matrix(y_test,y_pred))
ConfusionMatrixDisplay.from_predictions(y_test,y_pred)
plt.show()
```

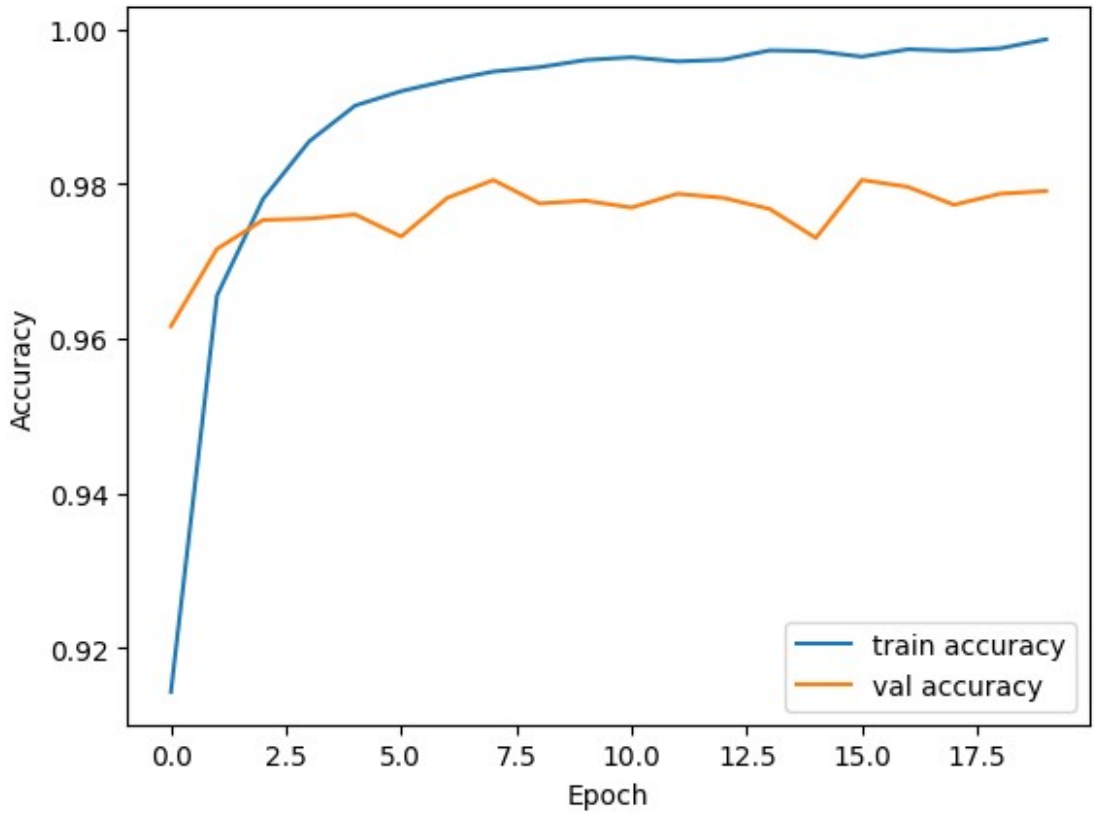
```
[[1328    1    4    0    0    0    2    3    5    0]
 [   0 1589    2    3    1    0    0    4    1    0]
 [   0    4 1359    1    1    2    1    4    6    2]
 [   0    0   18 1384    0   14    0    4   10    3]
 [   3    1    4    0 1266    0    4    4    3   10]
 [   0    3    0    7    1 1242    4    0   15    1]
 [   2    3    4    0    2    3 1370    0   12    0]
 [   1    1   14    0    3    1    0 1476    3    4]
 [   2    5   10    4    4    1    2    6 1320    3]
 [   4    4    3    4   16    4    0   13    9 1363]]
```




```
plt.plot(history.history["loss"],label="train loss")
plt.plot(history.history["val_loss"],label="val loss")
plt.xlabel("epochs")
plt.ylabel("loss")
plt.legend()
plt.show()
```




```
plt.plot(history.history["accuracy"], label="train accuracy")
plt.plot(history.history["val_accuracy"], label="val accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```


1.00
0.98
0.96
0.94
0.92
Accuracy
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Epoch
train accuracy
val accuracy